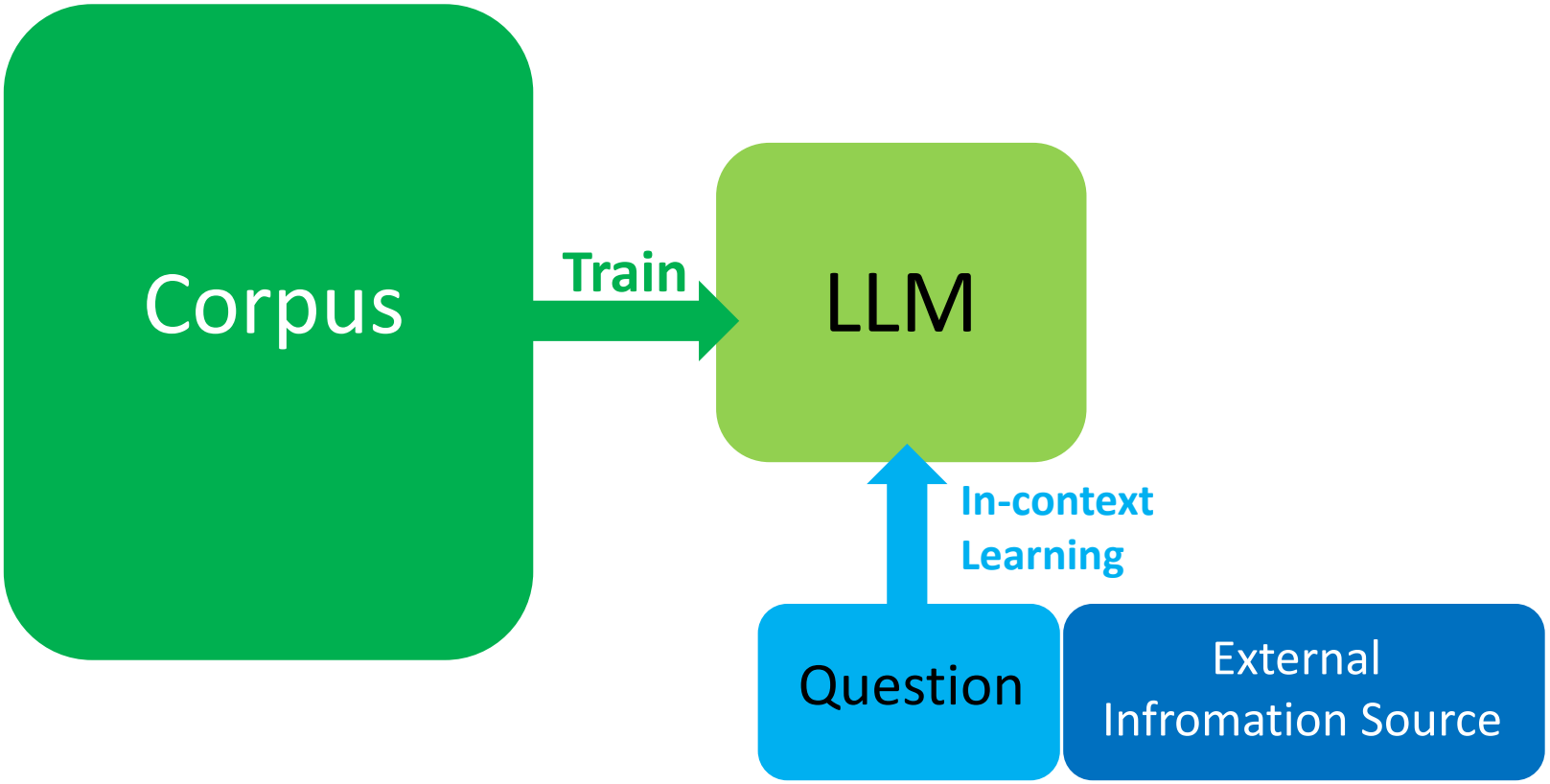
Assignment

Retrieval-Augmented Generation (RAG) Contest

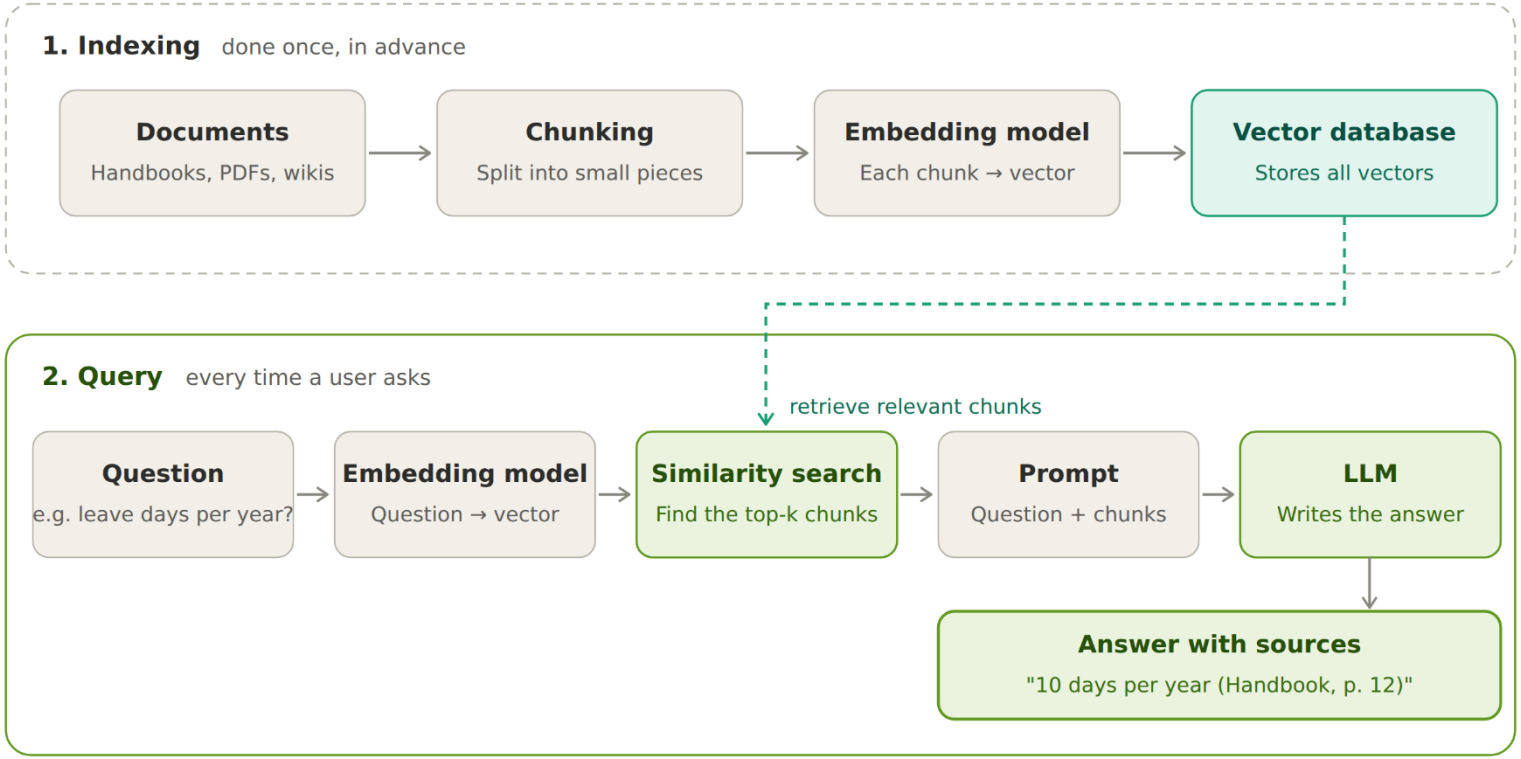
[30 points]

Retrieval-Augmented Generation (RAG)



Retrieval-Augmented Generation (RAG)

RAG (Retrieval-Augmented Generation) is a technique in which an LLM **first retrieves relevant information from an external source, then uses it to compose its answer**, rather than answering only from knowledge memorized in its weights.



Retrieval-Augmented Generation (RAG)

RAG solves problems with LLMs:

- **Outdated knowledge:** *the model doesn't know anything that happened after training.*
- **No private data:** *it can't see internal documents like handbooks or contracts.*
- **Hallucination:** *it makes things up when it doesn't know the answer.*
- **No sources:** *it can't show where its answers come from, so they're hard to check.*
- **Limited context:** *there's too much data to fit in one prompt, so RAG sends only the relevant parts.*

Assignment Summary

- Assignment นี้เป็นงานเดี่ยว คะแนนเต็ม 30 คะแนน
- ในงานนี้นักศึกษาจะต้องเขียนโปรแกรมเพื่อสร้างระบบ Question Answering (QA) โดยใช้ RAG กับ LLM
- โดยอาจารย์จะมี Document ให้ 150 ไฟล์ และในการทดสอบนักศึกษาต้องรันโปรแกรม QA ที่สร้างไว้ เพื่อตอบคำถาม 100 คำถาม โดยคะแนนจะขึ้นอยู่กับความถูกต้องของคำตอบที่ได้
- ข้อมูลที่เกี่ยวข้อง Post ไว้ใน Mango แล้ว

Documents

- Document ทั้ง 150 ไฟล์จะเป็น .pdf ขนาดรวมไม่เกิน 20 MB
- ในแต่ละไฟล์จะมีจำนวนหน้าไม่เท่ากัน แต่จะไม่เกินไฟล์ละ 5 หน้า โดยข้อมูลจะเป็น Text ล้วน ไม่มีภาพประกอบ
- Text ที่มีอาจจะมีในรูปแบบของตาราง หรือสมการได้บ้างในบางไฟล์
- Document จะถูกแจกจ่ายให้ล่วงหน้า 100 ไฟล์ โดยอีก 50 ไฟล์จะแจกให้ในวันทดสอบ (นักศึกษาจะไม่เห็นข้อมูล 50 ไฟล์นั้นก่อน)
- ภาษาที่ปรากฏใน Document จะเป็นภาษาอังกฤษเท่านั้น
- แต่ละ Document จะมีเลข ID (001-150) อยู่ด้านหน้าของชื่อไฟล์
- ข้อมูลที่ปรากฏใน Document ที่ให้เป็นเพียงข้อมูลที่แต่งขึ้นมา และอาจจะขัดแย้งกับข้อมูลในโลกแห่งความเป็นจริง

Questions

- คำถามจะมี 4 รูปแบบดังนี้ คือ
  1. **คำถามใช่หรือไม่** คำถามรูปแบบนี้จะมีคำตอบได้แค่ 2 แบบ คือ “yes” หรือ “no” เท่านั้น
  2. **คำถามตัวเลือก** คำถามรูปแบบนี้จะมีสูงสุด 6 ตัวเลือกให้เลือก ได้แก่ A B C D E F โดยให้ตอบเฉพาะตัวอักษรตัวใดตัวหนึ่งจาก A B C D E F เท่านั้น
  3. **คำถามตัวเลข** คำถามรูปแบบนี้จะต้องตอบมาเป็นตัวเลข Scalar ค่าหนึ่งเท่านั้น โดยจะต้องไม่มีข้อความหรือตัวอักษรใด ๆ ปนมาเลย
  4. **คำถามที่ตอบเป็นข้อความสั้น ๆ** คำถามรูปแบบนี้จะตอบออกมาเป็นข้อความอิสระ แต่คำตอบจะเป็นข้อความสั้น ๆ เท่านั้น เช่น ถามชื่อ, เวลา หรือ สถานที่ เป็นต้น

Questions

- ใน 100 คำถามทดสอบนั้น ก็จะมีแต่ละรูปแบบคละปะปนกันไป โดยแต่ละประเภทจะมีจำนวนที่ใกล้เคียงกัน
- ในการตอบแต่ละคำถามนั้นจะใช้ข้อมูลจาก Document เพียงไฟล์เดียวเท่านั้น (ไม่มีการใช้ข้อมูลข้ามไฟล์) และคำตอบจะหาได้จากข้อมูลใน Document ใด Document หนึ่งเสมอ โดยคำตอบที่ถูกต้องนั้นก็จะอ้างอิงจากข้อมูลใน Document เท่านั้น ไม่ใช่ข้อมูลจากโลกแห่งความเป็นจริง
- คำถามจะที่ให้จะเป็นภาษาอังกฤษเท่านั้น และมีความยาวไม่เกิน 100 คำ
- ทั้ง 100 คำถามจะถูกเขียนในไฟล์ .csv ที่มี 2 คอลัมน์ คือ question_id (1-100) และ question_text (ข้อความของคำถาม)
- ทั้ง 100 คำถามจะถูกแจกในวันทดสอบ แต่จะมีตัวอย่างคำถามอื่น ๆ ให้ไปลองใช้ทดสอบโปรแกรม (ดูไฟล์ question_samples.csv)

Answers

- ในการทดสอบนักศึกษาจะต้องสร้างไฟล์ .csv สำหรับตอบคำถามทั้ง 100 ข้อ โดยไฟล์นี้จะมี 3 column ดังนี้
    1. question_id ก็คือ ระบุว่าคำตอบใน row นั้นเป็นคำตอบของคำถามข้อที่เท่าไหร่ (เป็นไปได้ 1-100)
    2. answer_text ก็คือ คำตอบ (ขึ้นอยู่กับแต่ละรูปแบบคำถาม)
    3. document_id ก็คือ ให้ระบุว่าคำตอบที่ได้มานั้น มาจาก Document ID ที่เท่าไหร่ (เป็นไปได้ 1-150)
- ตัวอย่างรูปแบบไฟล์คำตอบดูได้ที่ answer_samples.csv

Answers

- การให้คะแนนคำตอบ ในแต่ละคำถามจะมีคะแนน 2 คะแนน โดยแบ่งออกเป็นดังนี้
  1. คะแนนความถูกต้องของ document_id ถ้าสามารถตอบ document_id ได้ถูกต้อง จะได้คะแนน 1 คะแนน
  2. คะแนนความถูกต้องของ answer_text เต็มอีก 1 คะแนน โดยคะแนนส่วนนี้จะคิดแตกต่างกันตามประเภทของคำถาม ดังนี้

คำถามใช่หรือไม่ และ คำถามตัวเลือก

$$score = \begin{cases} 1 & lowercase(\hat{a}) = lowercase(a) \\ 0 & otherwise \end{cases}$$

คำถามตัวเลข

$$score = \frac{1}{1 + |a - \hat{a}|}$$

คำถามที่ตอบเป็นข้อความสั้น ๆ

$$score = \frac{1}{1 + \frac{2 \times LD(a, \hat{a})}{length(a)}}$$

LD = Levenshtein distance

Levenshtein Distance

- **Edit distance** between two string/symbol sequence

- Levenshtein Distance $LD(X, Y)$ = Minimum no. of operations need for editing sequence X to Y

Operations

INSERTION : "**KARN**" ⇒ "**KARNY**", "ปก" ⇒ "ปาก"

DELETION : "**SPORT**" ⇒ "**SORT**", "ควาย" ⇒ "คาย"

SUBSTITUTION : "**3.98**" ⇒ "**1.98**", "สาย" ⇒ "สวย"

LD("anna", "banana") = 2 (2 insertion)

LD("bigger", "thinner") = 4 (3 substitution and 1 insertion)

Schedule

กิจกรรม	กำหนดการณ์	Assignment Score			Bonus
		Questionnaire	Rank	Milestone	
Com Spec Survey	16 Sept	1			
Question Examples	5 Oct (23:59)	7			
Development period 1	3 – 16 Oct				
Ranking Round (Room 521)	16 Oct (16.00-18.00)		6	16	6
Development period 2	17 Oct – 6 Nov				
Re-challenge Round (Online)	6 Nov (16.00-18.00)				

Question Examples

- ตอนนี้ตัวอย่างคำถาม (question_samples.csv) และคำตอบ (answer_samples.csv) ที่สามารถลองทดสอบความสามารถของโปรแกรมมีอยู่เพียง 8 คำถาม
- ยังต้องการขอให้นักศึกษาทุกท่านช่วยกันแต่งคำถามคำตอบมาอย่างน้อยคนละ 8 ตัวอย่าง โดยมีเงื่อนไขดังนี้
  - แต่งคำถามเป็นภาษาอังกฤษ
  - ต้องแต่งเองโดยที่ไม่ได้คัดลอกจากที่ไหนมา (ไม่ให้ลอกเพื่อน ห้าม AI ช่วยแต่ง แต่แต่งเองแล้วให้ AI ช่วยแปลหรือแก้ภาษาได้)
  - ใน 8 ตัวอย่างนี้ จะมีคำถามประเภทละอย่างน้อย 2 คำถาม
  - ขอให้เลือก Document ID ที่มีเลขลงท้ายเป็นเลขเดียวกับรหัสนักศึกษาเรา เช่น หากนักศึกษารหัสลงท้ายด้วยเลข 4 ขอให้ตั้งคำถามจาก Document 4, 14, 24, 34, 44, 54, 64, 74 ,84 หรือ 94 เท่านั้น
  - สามารถแต่งคำถามมากกว่า 1 คำถามจาก Document เดียวกันได้ แต่ขอให้กระจายคำถามไปอย่างน้อยใน 5 Document
  - คำถามและคำตอบจะต้องไม่กำกวมและหาข้อมูลได้จากใน Document นั้น
- ขอให้ส่งประโยคใน Google Form ด้านล่างนี้ ภายในวันที่ 5 ต.ค. 69 (23.59)

https://forms.gle/fPLgafnrVCKXhBDE7

Rewards (Question Examples)

จำนวนตัวอย่างคำถามคำตอบที่ส่งมา	Assignment Score	พ่อง
ส่งส่งช้าหรือผิดเงื่อนไข	(1,7)	
8	7	
12	7	200,000

Testing

- การทดสอบโปรแกรมจะมีอยู่ 2 รอบ คือ รอบที่ 1 เรียกว่า Ranking Round และ รอบที่ 2 เรียกว่า Re-challenge Round
- แต่ละรอบจะมีคำถามทดสอบจำนวน 100 คำถาม ซึ่งเกือบทั้งหมดจะเป็นคำถามที่ไม่ได้ให้ไป โดยรอบที่ 1 กับ 2 ก็จะเป็นคำถามคนละชุดกัน
- ก่อนแจกคำถามทดสอบ นักศึกษาจะต้องส่ง source code ที่เกี่ยวข้องกับการพัฒนาโปรแกรมทั้งหมด โดยเมื่อแจกประโยคทดสอบแล้วจะไม่อนุญาตให้แก้ไขโปรแกรมแล้ว (หากมีเหตุจำเป็นจริง ๆ จะพิจารณาเป็นกรณีไป)
- โปรแกรมที่ส่งต้องมีเวอร์ชันเดียว และการทดสอบจะต้อง Run เวอร์ชันนั้น นักศึกษาไม่สามารถเขียนมาหลายเวอร์ชันแล้วเลือกส่งได้
- ในการทดสอบทั้ง 2 รอบ โปรแกรมจะต้องรันบน Google Colab โดย Set Run-time Type เป็น CPU และในการทำงานของโปรแกรมต้องใช้ System RAM ไม่เกิน 12.7 GB และ Run เสร็จภายในเวลาที่กำหนด (เวลา 18.00)
- ในกรณีที่การใช้งาน Google Colab มีปัญหา สามารถ Run บนเครื่องตนเองได้ โดยโปรแกรมต้อง Run โดยไม่ใช้ GPU และ ใช้ RAM ไม่เกินที่กำหนด (อาจารย์จะลองเอา Code ทุกคนที่ส่งไป Run เองใน Colab ภายหลัง)
- ก่อนหมดเวลา นักศึกษาจะต้องส่ง “answer.csv” ที่ได้จากโปรแกรม โดยที่หาก อ. Run ซ้ำอีกครั้งหนึ่ง ต้องได้ “answer.csv” ตัวเดิม

Rewards (Rank)

- รางวัลส่วนที่จะพิจารณาจากค่า Score ที่ได้จากรอบที่ 1 Ranking Round เท่านั้น
- หาก Score เสมอกัน การพิจารณาลำดับจะพิจารณาจากลำดับการส่งคำตอบ (ส่งก่อนอันดับจะสูงกว่าถ้า Score เท่ากัน)

Rank จาก Ranking Round	Assignment Score	พ่อง
1	6	1,000,000
2	5.5	900,000
3	5	800,000
4-5	4.5	700,000
6-10	4	600,000
11-15	3.5	500,000
16-20	3	400,000
21-25	2.5	300,000
26-30	2	200,000
31-40	1.5	100,000
>=41	1	0
โปรแกรมรันไม่ได้ หรือไม่ได้มาทดสอบ	0	0

Rewards (Milestone)

- รางวัลส่วนที่จะพิจารณาจากค่า Score ที่ได้จากรอบที่ 1 Ranking Round และรอบที่ 2 Re-challenge โดยถือเอา Score รอบที่ดีที่สุดมาใช้พิจารณา
- สำหรับ Bonus (คะแนนและพ่อง) ในแต่ละระดับ หากมีคนทำได้มากกว่า 1 คนในระดับนั้น Bonus จะถูกหารกันสำหรับคนที่ทำได้
- Bonus ที่ถูกทำได้ในรอบ Ranking Round จะถูกเอาออกไปในรอบ Re-challenge แต่จะมีการเพิ่ม Bonus ระดับที่สูงกว่าเข้ามาแทนในรอบ Re-challenge

Score ที่ดีที่สุด	Assignment Score	Bonus Score	Bonus พ่อง
<120	$\frac{score}{7.5}$		
>=120	16		
>=130	16		200,000
>=140	16		200,000
>=150		1	200,000
>=160		1	500,000
>=170		2	500,000
>=180		2	1,000,000

Program Development

- นักศึกษาจะต้องพัฒนาโปรแกรมที่รับ question.csv และสร้าง answer.csv ที่เป็นคำตอบออกมา
- งานนี้เป็นงานเดี่ยว นักศึกษาต้องพัฒนาโปรแกรมด้วยตัวเอง แต่สามารถใช้ AI ในการช่วยพัฒนาได้
- นักศึกษาต้องพัฒนาโปรแกรมนี้ด้วยภาษา Python สามารถพัฒนาบนคอมพิวเตอร์เครื่องไหนก็ได้ แต่ตอนทดสอบโปรแกรมต้องสามารถ Run ได้บน Google Colab โดยไม่ใช้ GPU (ใช้แต่ CPU) และใช้ System RAM ไม่เกิน 12.7 GB
- การคำนวนทั้งหมดต้องทำบน Local กล่าวคือ การแปลง Vector หรือ การประมวลผลของ LLM ทุกอย่างจะต้องทำการคำนวนโดยการโหลด Weight เข้า System Memory แล้วคำนวณบนเครื่อง โดยไม่มีการส่งข้อมูลผ่านอินเตอร์เน็ตเพื่อไปประมวลผลภายนอก นอกเหนือจากการอ่านข้อมูลจาก Google Drive ของตัวเอง โปรแกรมจะเรียกข้อมูลจากอินเตอร์เน็ตได้เฉพาะตอน Install Library และ Load Model เท่านั้น
- นักศึกษาสามารถเรียกใช้ LLM ที่ทำหน้าที่ Text Generation ได้เพียง Model เดียวเท่านั้น โดย Model นั้นจะต้องมีขนาดไม่เกิน 2 พันเล้านพารามิเตอร์ (2B) แต่ในการตอบคำถาม 1 ข้อ สามารถเรียกใช้ Model นี้มากกว่า 1 ครั้งได้
- นักศึกษาสามารถใช้วิธีการใดในการทำ Document Retrieval และ Vector Embedding ก็ได้ตราบใดที่ไม่ผิดเงื่อนไขอื่น ๆ (ในโปรแกรมจะไม่มีขั้นตอนที่ทำ Document Retrieval และ Vector Embedding ก็ได้)
- นักศึกษาสามารถเลือกใช้เทคนิคใดก็ได้ในการพัฒนาโปรแกรมทั้งที่เคยสอนหรือไม่เคยสอน ตราบใดที่ไม่ผิดเงื่อนไขอื่น ๆ